

การเปรียบเทียบสมรรถนะกับงานวิจัยที่ตีพิมพ์แล้ว

CyberCase Intelligence Framework เทียบกับ TechniqueRAG, H-TechniqueRAG และ TTP-LM บนชุดข้อมูลและตัวชี้วัดเดียวกัน

สาขาที่วัด: การระบุเทคนิค MITRE ATT&CK จากข้อความข่าวกรองภัยคุกคาม · branch `eval/published-baselines`

1. เหตุผลและที่มา

อาจารย์ที่ปรึกษาเสนอว่า การวัดผลระบบด้วยชุดข้อมูลที่สร้างขึ้นเองไม่เพียงพอต่อการยืนยันคุณภาพ เพราะไม่มีจุดอ้างอิงภายนอก จึงควรทดสอบด้วย ชุดข้อมูลและตัวชี้วัดชุดเดียวกับงานวิจัยที่ตีพิมพ์แล้ว เพื่อให้ตัวเลขของเราอยู่ในตารางเดียวกับเขาและตีความได้ทันที

งานวิจัยที่เลือกเป็นคู่เทียบ

ชื่อย่อ	งานวิจัย	ชุดข้อมูล	ตัวชี้วัด
TTP-LM	Advancing TTP Analysis: Encoder-Only and Decoder-Only LMs with RAG (arXiv 2401.00280)	MITRE procedure examples	Precision, Recall
TechniqueRAG	TechniqueRAG: Retrieval Augmented Generation for Adversarial Technique Annotation (ACL Findings 2025, arXiv 2505.11988)	TRAM, Procedures, Expert	P, R, F1, P@k / R@k (k=1,3)
H-TechniqueRAG	Hierarchical RAG for Adversarial Technique Annotation (arXiv 2604.14166)	CTI-RCM, MITRE CTI, TRAM	micro-F1, P, R, MAP@10, latency

ทั้งสามงานทำโจทย์เดียวกัน คือรับข้อความ CTI ภาษาอังกฤษสั้น ๆ แล้วคืนชุดรหัสเทคนิค ATT&CK และ **TRAM** เป็นชุดข้อมูลรวมของทั้งสามงาน จึงถูกเลือกเป็นเป้าหมายหลัก — วัดครั้งเดียวเทียบได้ทั้งสามงาน

ขอบเขตที่ต้องระบุให้ชัด

โจทย์ของเขาเป็นการติดป้ายกำกับข้อความสั้น (80–400 ตัวอักษร) ส่วนโจทย์ของเราคือ **สำนวนคดีภาษาไทยขนาดยาว** → **รายงานสรุปเชิงโครงสร้างสำหรับพนักงานอัยการ** สิ่งที่ยกกันได้จริงคือส่วนการค้นคืนและการระบุเทคนิค ไม่ใช่ผลผลิตปลายทาง ข้อจำกัดนี้ต้องระบุทุกครั้งให้อ้างตัวเลข

นอกจากนี้ **ทุก arm ของเราเป็น zero-shot** ไม่ได้ฝึกกับ train split ของชุดข้อมูลเหล่านี้ ขณะที่ตัวเลขพาดหัวของ TechniqueRAG มาจากโมเดลที่ fine-tune แล้ว การเทียบที่เป็นธรรมจึงต้องเทียบกับแถว zero-shot และแถว retriever สำเร็จรูปของเขา

2. การเตรียมข้อมูล

2.1 ชุดข้อมูล

ดึงจาก HuggingFace QCRI/TechniqueRAG-Datasets เฉพาะ split `*_zeroshot_test` เพราะ split แบบ few-shot ผัง context ที่ค้นมาแล้วไว้ใน prompt ซึ่งเป็นส่วนที่ระบบเราต้องผลิตเอง ใช้วิธีดึงตอนรัน ไม่ผนวกเข้า repo เนื่องจากโค้ดต้นทางเป็นสัญญาอนุญาต GPL-3.0

ชุดข้อมูล	จำนวน	ป้ายกำกับ	ความยาว input (median)	สัดส่วน sub-technique
TRAM	725	single-label (87%)	95 ตัวอักษร	23.9%
Expert	157	multi-label (median 3)	406 ตัวอักษร	39.3%
Procedures	1,767	single-label	80 ตัวอักษร	50.7%

2.2 การพิสูจน์ว่าตัวชี้วัดตรงกับต้นทาง

การเขียนสูตร P/R/F1 ขึ้นเองไม่เพียงพอ เพราะ F1 มีหลายรูปแบบและต่างกันเล็กน้อยก็เทียบไม่ได้ จึงนำฟังก์ชันจาก `evaluate.py` ของต้นทางมาเป็นตัวอ้างอิง (เก็บไว้ใน repo เพื่อไม่ให้ปนสัญญาอนุญาต GPL) แล้วป้อนข้อมูลสุ่ม 400 ชุดเข้าทั้งสองฝั่ง

ผล: Precision, Recall และ F1 ตรงกันทุกชุด คลาดเคลื่อนไม่เกิน 1e-12 ทั้งสองโหมด

ข้อสังเกตของโปรโตคอลต้นทางที่ต้องทำตามให้ตรง มิฉะนั้นตัวเลขจะเทียบไม่ได้

ประเด็น	พฤติกรรมของต้นทาง	การจัดการ
การคำนวณ F1	คำนวณจากค่าเฉลี่ยของ P และ R ไม่ใช่เฉลี่ย F1 รายตัวอย่าง	ทำตามทุกประการ และรายงาน micro-F1 เพิ่มเป็นตัวเทียบ
โหมดปริยาย	<code>--mode technique</code> ยูน sub-technique ขึ้นเป็น parent	รายงานทั้งสองโหมด และระบุกำกับเสมอว่าตัวเลขมาจากโหมดใด
MRR	คำนวณหลัง <code>list(set(preds))</code> ซึ่งทำลายลำดับ และ Python สุ่ม hash ของ string ต่อ process จึงให้ค่าไม่คงที่ระหว่างการรัน	รายงาน MRR จากลำดับจริง พร้อมระบุว่าตัวชี้วัดนี้เทียบตรงไม่ได้

2.3 การปรับรุ่น ATT&CK ให้ตรงกัน — ขั้นตอนที่สำคัญที่สุด

ป้ายเฉลี่ยของชุดข้อมูลถูกกำกับไว้ในยุค ATT&CK ราวรุ่น v14 ขณะที่ดัชนีของเราสร้างจาก **v19.0** ในช่วงเวลาระหว่างนั้น MITRE ได้ยกเลิกและย้ายรหัสจำนวนหนึ่ง หากไม่ปรับให้ตรงกันก่อน ระบบจะค้นเจอสิ่งเดียวกันแต่ถูกนับว่าผิดเพราะรหัสไม่ตรง ซึ่งเป็นการวัดที่ผิด ไม่ใช่ระบบที่แย่

รหัสในชุดข้อมูล (v14)	สถานะใน v19	ผลลัพธ์
T1562 Impair Defenses	ย้ายไป T1685	แปลงรหัส
T1562.001 – .006	ย้ายไป T1685 / T1685.001 / T1686 / T1690	แปลงรหัส
T1574.002 DLL Side-Loading	ยุบรวมเป็น T1574.001 (DLL)	แปลงรหัส
T1070.001 / .002	ย้ายไป T1685.005 / T1685.006	แปลงรหัส
T1043, T1064	ยกเลิกโดยไม่มีรหัสทดแทน	ตัดตัวอย่างทิ้ง (12 ตัวอย่าง)

ตารางแปลงรหัสสร้างขึ้นจากความสัมพันธ์ `revoked-by` ในไฟล์ STIX ของ MITRE ที่มีอยู่ใน repo อยู่แล้ว ได้ทั้งสิ้น **196 คู่** และรหัสที่ตายสนิท 25 รายการ จึงเป็นกระบวนการที่ทำซ้ำได้ ไม่ใช่การแก้ด้วยมือ

ผลหลังการแปลง: ป้ายเฉลี่ยทุกรายการในทั้งสามชุดข้อมูลค้นคืนได้จากดัชนีของเราครบ **100%** หมายความว่าคะแนนที่เสียไปหลังจากนี้ เสียเพราะค้นไม่เจอจริง ไม่ใช่เพราะไม่มีข้อมูลให้ค้น TRAM เหลือตัวอย่างที่วัดได้ **713 จาก 725**

3. การพัฒนาชุดทดสอบ

3.1 โครงสร้าง

ทั้งหมดอยู่ที่ `rag_service/app/RAG/GraphRAG/evaluation/published/`

ไฟล์	หน้าที่
<code>fetch_datasets.py</code>	ดึง split ทดสอบจาก HuggingFace พร้อมตรึงชื่อ repo และไฟล์
<code>metrics.py</code>	ตัวชี้วัดตามโปรโตคอลต้นทาง พร้อม unit test 18 รายการ
<code>attack_version_map.py</code>	สร้างและใช้ตารางแปลงรหัส ATT&CK v14 → v19
<code>run_benchmark.py</code>	ตัวรันทั้ง 5 arm รองรับการรันต่อจากจุดที่ค้าง (resume)
<code>BASELINES.md</code>	ตัวเลขที่ตีพิมพ์ของงานคู่เทียบ ไว้อ่านประกอบผลของเรา
<code>NOTICE.md</code>	ที่มาของข้อมูล สัญญาอนุญาต และข้อจำกัดที่ต้องระบุในรายงาน

3.2 arm ที่ใช้วัด และหลักการแยก headline ออกจาก ablation

หลักการที่ยึด: ตัวเลขที่อ้างว่าเป็นคะแนนของระบบ ต้องมาจาก **pipeline** เดียวกับที่ใช้จริงบน **production** arm อื่นมีไว้เพื่ออธิบายว่าทำไมตัวเลขนั้นถึงออกมาเช่นนั้น ไม่ใช่เพื่อนำไปอ้างแทน

arm	retrieval	LLM	สถานะ	ใช้ตอบคำถาม
agent	ครบ	ครบ	headline	ระบบที่ให้บริการจริงทำได้เท่าไร (router + decompose + evaluator + loop)
retrieval	มี	ไม่มี	ablation	คุณภาพการค้นคืนสั้นๆ
rag-en	มี	มี	ablation	ค้นคืนแล้วให้ LLM สรุป โดยตัด router / evaluator / loop ออก
llm-only	ไม่มี	มี	ablation	โมเดลตอบเองได้เท่าไรโดยไม่มีเรา (เส้นฐาน)
rag-th	มี	มี	ไม่ได้รัน	ผลของการเขียนคำตอบเป็นภาษาไทย

เหตุผลที่ไม่รัน **rag-th**: input ของ TRAM เป็นภาษาอังกฤษทั้งหมด arm นี้จึงเป็น “input อังกฤษ → output ไทย” ซึ่งไม่ใช่เส้นทางจริงของระบบ (ไทย → ไทย) และไม่ได้ทดสอบ BGE-M3 กับภาษาไทยเลย เพราะข้อความที่ป้อนเข้า retriever ยังเป็นอังกฤษเหมือนเดิม การวัดแกนภาษาไทยควรทำบนชุดข้อมูล CTI ไทยของโครงการเอง

3.3 การตั้งค่าที่ใช้

โมเดลภาษา `openai/gpt-5.6-luna` ผ่าน OpenRouter (temperature 0) · embedding `BAAI/bge-m3` · reranker `BAAI/bge-reranker-v2-m3` · vector top-K = 20 · คั่นเฉพาะโหนด Technique / Subtechnique

3.4 ข้อบกพร่องที่พบและแก้ระหว่างการพัฒนา

ข้อบกพร่อง	อาการ	การแก้ไข
node_label_filter ไม่ทำงาน	retrieve() รับพารามิเตอร์นี้และระบุไว้ใน docstring แต่ไม่ส่งต่อ และ search_all() ไม่มีพารามิเตอร์นี้ ผู้เรียกจึงได้ผลลัพธ์ที่ไม่ถูกกรองโดยไม่มีการแจ้งเตือน	ต่อพารามิเตอร์ให้ครบทั้งสาย และรองรับหลาย label พร้อมกัน
Qdrant ปฏิเสธการกรอง	การกรองฝั่งเซิร์ฟเวอร์ล้มเหลวทั้ง 713 ตัวอย่าง ด้วย 400 Index required but not found เนื่องจาก collection ไม่มี payload index และเปิด strict mode	เปลี่ยนไปกรองฝั่งไคลเอนต์ (ตั้งเผื่อแล้วคิด) ตามรูปแบบที่โค้ดเดิมใช้กับ domain อยู่แล้ว เพื่อไม่ต้องแก้ไขฐานข้อมูลที่ใช้ร่วมกัน
context ถูกตัดเหลือ 5 รายการ	get_context_text() จำกัดที่ FINAL_TOP_K = 5 ทำให้ LLM เห็น candidate เพียง 5 จาก 20 รายการ	สร้างตัวประกอบ context เฉพาะสำหรับการวัดผล ให้ LLM เห็น candidate ครบทุกรายการ

4. ผลการวัด

ชุดข้อมูล TRAM · 713 ตัวอย่าง · zero-shot ทุก arm · ไม่มีข้อผิดพลาดระหว่างรัน

4.1 ผลของระบบเรา — ระดับ technique

arm	P	R	F1	P@1	P@3	R@3
llm-only (ไม่มี retrieval)	0.653	0.738	0.693	0.687	0.653	0.727
rag-en (มี retrieval)	0.589	0.665	0.625	0.621	0.589	0.663
retrieval — กรองเฉพาะ technique	0.070	0.805	0.129	0.373	0.251	0.551
retrieval — ไม่กรอง	0.145	0.667	0.238	0.384	0.263	0.548

4.2 ผลของระบบเรา — ระดับ sub-technique

arm	P	R	F1	P@1	P@3
llm-only	0.484	0.551	0.515	0.512	0.485
rag-en	0.429	0.495	0.460	0.454	0.429
retrieval — กรองเฉพาะ technique	0.038	0.695	0.073	0.264	0.152
retrieval — ไม่กรอง	0.086	0.542	0.149	0.261	0.155

4.3 เียบกับตัวเลขที่ตีพิมพ์ — TRAM ระดับ technique

ระบบ	ประเภท	P	R	F1
NCE	retriever	0.903	0.789	0.842
BM25	retriever	0.679	0.647	0.663
TechniqueRAG re-ranker	retriever	0.647	0.614	0.630
RankGPT	retriever	0.619	0.586	0.602
ของเรา — retrieval (P@1)	retriever	0.373	—	—
GPT-4o	zero-shot	0.383	0.500	0.434
DeepSeek v3	zero-shot	0.437	0.657	0.525
ของเรา — llm-only	zero-shot	0.653	0.738	0.693
GPT-4 (RAG)	RAG	0.555	0.706	0.622
DeepSeek v3 (RAG)	RAG	0.546	0.774	0.640
IntelEx	RAG	0.607	0.707	0.653
ของเรา — rag-en	RAG	0.589	0.665	0.625
TechniqueRAG	fine-tuned	0.760	0.721	0.740

หมายเหตุ: แถว retriever ของงานตีพิมพ์รายงานค่า $P \approx R$ บนชุด single-label ซึ่งบ่งชี้ว่าให้คำตอบบรรทัดหนึ่งรายการต่อตัวอย่าง ค่าที่เทียบกันได้จากฝั่งเรา จึงเป็น P@1 ไม่ใช่ precision ของรายการทั้งลิสต์ อนึ่ง TechniqueRAG ผ่านการ fine-tune บน train split จึงไม่ใช่คู่เทียบโดยตรงของ arm โดยของเรา

5. สรุปผล

ข้อค้นพบที่ 1 — ระบบค้นคืนของเราอ่อนในฐานะตัวจัดอันดับ แต่แข็งในฐานะตัวสร้างตัวเลือก

P@1 อยู่ที่ 0.373 ต่ำกว่าทุก baseline ที่ตีพิมพ์ แต่เมื่อพิจารณาทั้งลิสต์ 20 อันดับ เฉลยปรากฏอยู่ใน 83.3% ของตัวอย่าง (recall 0.805) แปลว่าปัญหาไม่ใช่การค้นไม่เจอ แต่เป็นการเรียงลำดับผิด

สาเหตุเชิงโครงสร้างที่ต่างจากเขา: corpus ของ TechniqueRAG คือบทสรุปรายเทคนิคที่เขียนขึ้นใหม่ และตัวอย่างที่มีเฉลยจาก **train split** ขณะที่ของเราเป็นคำอธิบาย STIX ดิบจากกราฟความรู้ทั้งกราฟ ซึ่งเป็นกองข้อมูลที่คั่นยากกว่ามากสำหรับข้อความชนิดนี้

ข้อค้นพบที่ 2 — การกรองให้เหลือเฉพาะ **technique** ไม่ช่วยเรื่องการจัดอันดับ

สมมติฐานเริ่มต้นคือผลลัพธ์ประเภท Group / Software แย่งตำแหน่งต้นตาราง แต่เมื่อวัดจริง P@1 แทบไม่เปลี่ยน (0.373 เทียบกับ 0.384) เพราะกลไกถ่วงน้ำหนักตามชนิดไหนได้ต้น technique ขึ้นก่อนอยู่แล้ว การกรองให้ผลเฉพาะด้าน **ความลึก** คือ recall เพิ่มขึ้นจาก 0.667 เป็น 0.805

ข้อค้นพบที่ 3 — การเพิ่ม **retrieval** ทำให้คำตอบแย่ลง

เป็นข้อค้นพบที่สำคัญที่สุดของการทดลองนี้ การนำ context ที่ค้นได้ไปประกอบคำตอบ ทำให้ F1 ลดลง **6.8 จุด** (0.693 → 0.625) และลดลงทั้ง precision และ recall จึงไม่ใช่การแลกเปลี่ยนระหว่างสองค่า

จากการจับคู่ผลลัพธ์รายตัวอย่าง

- 93 ตัวอย่างที่ llm-only ตอบถูกแต่ rag-en ตอบผิด ขณะที่ทิศทางกลับกันมีเพียง 37 ตัวอย่าง
- **66%** ของตัวอย่างที่เสียไป เฉลยไม่ได้อยู่ในรายการ **candidate** เลย แต่โมเดลยังตอบตาม candidate อันดับหนึ่ง
- 47% ของคำตอบจาก rag-en คือ candidate ที่ผิด

```
gold      : T1105 (Ingress Tool Transfer)
llm-only  : T1105                      ถูก
rag-en    : T1588.001                  ตอบตาม candidate อันดับ 1
cands     : T1588.001, T1102.001, T1095, ... (ไม่มี T1105)
```

กลไก: ตัวค้นคืนที่แม่นยำอันดับหนึ่งเพียง 0.373 กำลังตรึงความคิด (anchoring) โมเดลที่ตอบเองได้ 0.687 ประกอบกับคำสั่งใน prompt ที่ระบุให้ “ใช้ข้อมูลที่ค้นมาเป็นหลักฐาน” ยิ่งบังคับให้เชื่อตัวเลือกที่ผิด

ข้อค้นพบที่ 4 — ข้อควรระวังเรื่องการปนเปื้อนของชุดข้อมูล

arm llm-only ทำคะแนนได้ 0.693 ซึ่งสูงกว่าแถว zero-shot ของงานตีพิมพ์อย่างมาก (GPT-4o 0.434, DeepSeek v3 0.525) และเกือบเท่าตัวเลขพาดหัวของ **TechniqueRAG** ที่ผ่านการ **fine-tune** แล้ว (0.740)

procedure examples ของ MITRE เป็นข้อมูลสาธารณะ และโมเดลที่ใช้เป็นรุ่นปี 2026 จึงมีความเป็นไปได้สูงว่าโมเดลเคยเห็นข้อมูลชุดนี้ในระหว่างการฝึก หากเป็นเช่นนั้น **ชุดทดสอบนี้ไม่สามารถแสดงคุณค่าของการค้นคืนได้เลยไม่ว่าจะเป็นระบบใด** ข้อนี้เป็นเหตุผลสนับสนุนการใช้ชุดข้อมูล CTI ภาษาไทยที่โครงการจัดทำขึ้นเอง มิใช่ข้อแก้ตัวของผลที่ได้

6. ข้อจำกัดของการทดลองนี้

- ตัวเลขทั้งหมดในรายงานฉบับนี้มาจาก arm ที่เป็น **ablation** ยังไม่ใช่ตัวเลขของ pipeline ที่ให้บริการจริง arm agent ได้พัฒนาเสร็จแล้วแต่ยังไม่ได้รัน
- วัตนชุด TRAM เท่านั้น ยังไม่ได้วัด Expert (multi-label) และ Procedures

- การเทียบ rag-en กับแนว RAG ของงานตีพิมพ์ ปนตัวแปรสองอย่างคือตัวค้นคืนที่ต่างกันและโมเดลภาษาที่ต่างกัน แยกออกจากกันไม่ได้
- MRR ไม่สามารถเทียบกับต้นทางได้ ด้วยเหตุผลที่ระบุในหัวข้อ 2.2

7. งานที่ควรทำต่อ

ลำดับ	งาน	เหตุผล	ต้นทุนโดยประมาณ
1	รัน arm agent บนชุด Expert	ได้ตัวเลขของ pipeline จริงตามหลักการ และ Expert เป็น split เดียวที่มี P@1 / P@3 ที่ตีพิมพ์ให้เทียบตรง	~45 นาที · ~\$0.30
2	ทดลองแก้ข้อผิดพลาดใน prompt	เปลี่ยนจาก “ใช้ข้อมูลที่ค้นหาเป็นหลักฐาน” เป็น “ใช้เมื่อเป็นประโยชน์” เพื่อแยกค่าที่เสีย 6.8 จุด เกิดจาก prompt ที่บังคับ หรือจากคุณภาพ candidate — หากเป็นอย่างแรก ก็ได้เห็นหีบ production	~20 นาที · ~\$0.35
3	ทดสอบผลของการแตกคำถามย่อยบนชุด Expert	TechniqueRAG แตกคำถามย่อยอยู่ใน prompt ของ re-ranker การไม่ทำจึงเป็นการเสียเปรียบบน split ที่เป็น multi-label	~15 นาที · ~\$0.05
4	แก้การรวมคะแนนข้าม collection	การ normalize แบบ min-max แยกตาม collection บังคับให้อันดับหนึ่งของแต่ละ collection มีค่าเท่ากับ 1.0 เสมอ ไม่ว่าจะเกี่ยวข้องจริงหรือไม่ กระบวนการค้นคืนของ production ทั้งระบบ	ต้องวัดก่อนและหลัง
5	พิจารณา fine-tune เพื่อเทียบกับตัวเลข 0.740	โครงการมีโมเดล fine-tune อยู่แล้ว การฝึกบน train split ของ TRAM จะทำให้เทียบกับตัวเลขพาดหัวของเขาได้อย่างเป็นธรรม	เป็นงานแยกอีกก้อน

ข้อมูลประกอบทั้งหมด ได้แก่ ผลการรันรายตัวอย่าง ที่มาของข้อมูล และตัวเลขของงานคู่เทียบ อยู่ใน `rag_service/app/RAG/GraphRAG/evaluation/published/` บน branch `eval/published-baselines`